

FINE-TUNING, LoRA & QLoRA

From Scratch - Interview + Teaching Point of View

The main idea: A pretrained LLM already has general language ability. A task-specific dataset gives examples of the behavior we want. Fine-tuning uses those examples to calculate loss and update trainable parameters. LoRA trains small low-rank adapter matrices instead of changing the whole model. QLoRA additionally keeps the frozen base model in a low-bit, typically 4-bit, representation.

What you will learn

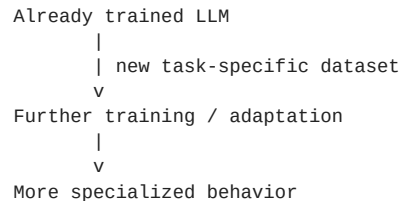
- What an LLM is: parameters, weights, tokens, embeddings and transformers
- Pretraining vs fine-tuning vs inference
- The exact role of a dataset: why we need it and what actually changes
- Supervised Fine-Tuning and next-token prediction
- Full fine-tuning, PEFT and LoRA
- LoRA mathematics: Delta W, A, B, rank and alpha
- Quantization, INT8, 4-bit and NF4
- QLoRA, double quantization and paged optimizers
- Optimizer memory, gradient checkpointing and gradient accumulation
- Llama + Unsloth style code flow
- Real-world system design and RAG vs fine-tuning
- Interview answers, common mistakes and a teaching script

One line to remember: Dataset = teacher/examples. Model = learner. Loss = feedback. Gradients = direction of change. Trainable parameters = what actually gets updated.

1. The Most Important Question: If the LLM Is Already Trained, What Are We Doing?

This is the central concept behind all of today's questions. Suppose we already have Llama, Qwen or Mistral. The model has already been pretrained. So why take another dataset? Because the pretrained model is general-purpose, while your application may need specific behavior, style, task performance or domain adaptation.

The correct picture



Are we training the dataset?

No. The dataset is not trained. The dataset is the training signal. It contains examples that let us measure how well the model is doing and calculate gradients. Those gradients are used to update trainable model parameters.

What does the dataset contain?

For supervised fine-tuning, a dataset commonly contains an input/instruction and a desired response. For example: 'Explain binary search' paired with a high-quality target answer.

Dataset example:

Input:
Explain binary search.

Target:
Binary search is an algorithm that searches a sorted collection by repeatedly dividing the search interval...

The complete learning loop

```
Dataset
-> input + target
-> model prediction
-> compare prediction with target
-> loss
-> backpropagation
-> gradients
-> optimizer update
-> repeat
```

Interview sentence: 'We fine-tune the model on a dataset; we do not fine-tune the dataset. The dataset provides examples, the loss provides feedback, and trainable parameters are updated.'

2. What Is an LLM?

2.1 Model = learned mathematical function

A large language model is a neural network with a very large number of learned numerical parameters. A parameter is simply a value that training adjusts so that the model becomes better at its objective.

Tiny illustration:

```
W = [[0.2, -0.7],  
     [0.5, 0.1]]
```

Real LLMs contain millions or billions of such learned values.

2.2 What does '3B' mean?

3B means approximately 3 billion parameters. The number of parameters is not the same thing as the amount of GPU memory required during training. Memory also depends on datatype, gradients, optimizer states, activations and runtime overhead.

2.3 What happens when you type a question?

```
Text  
|  
Tokenizer  
|  
Token IDs  
|  
Embeddings  
|  
Transformer blocks  
|  
Logits over vocabulary  
|  
Probabilities  
|  
Next token  
|  
Repeat generation
```

2.4 Tokenization

A tokenizer converts text into tokens and numerical token IDs. A token may be a word, part of a word, punctuation or another subword unit depending on the tokenizer.

2.5 Embeddings

Token IDs are indices. The model maps them to dense vectors called embeddings. Those vectors carry learned representations that the transformer processes.

2.6 Transformer blocks

At interview level, explain that transformer blocks contain self-attention, feed-forward/MLP layers, normalization and residual connections. Attention allows token representations to interact with contextual information.

```
Q = X W_Q  
K = X W_K  
V = X W_V
```

```
Attention(Q,K,V) =  
softmax(Q K^T / sqrt(d_k)) V
```

3. Pretraining: How the LLM Became 'Already Trained'

Pretraining is the initial large-scale learning stage. The model sees huge amounts of text and learns to predict tokens. The important point is that the weights change during this process.

Example:

Input: The capital of India is
Target: Delhi

Model -> probability distribution
-> calculate loss
-> backpropagate
-> optimizer updates parameters

3.1 The parameter update

$$W_{\text{new}} = W_{\text{old}} - \eta * \text{grad}_W(L)$$

W = model parameters
 L = loss
 grad = derivative of loss with respect to W
 η = learning rate

3.2 Why does the model learn?

If the model repeatedly receives contexts and target tokens, the optimization process changes the parameters so that correct target tokens tend to receive higher probability in similar contexts.

3.3 Pretraining vs fine-tuning

Aspect	Pretraining	Fine-tuning
Starting point	New/initial model	Already pretrained model
Dataset	Huge and broad	Smaller and focused
Goal	General capability	Specific adaptation
Cost	Very high	Lower, especially with PEFT
Typical result	Foundation model	Specialized/adapted model or adapter

Do not say: 'Fine-tuning means training the LLM from scratch.' **Say:** 'Fine-tuning is additional training of a pretrained model.'

4. Supervised Fine-Tuning (SFT)

4.1 Why SFT?

SFT is used when we have examples of the desired behavior. The dataset gives an input/context and a target response. For causal language models, the training objective is usually next-token prediction over the target sequence.

Example:
Instruction: Explain recursion.
Target: Recursion is a technique in which a function...

4.2 What happens to one example?

1. Tokenize the example
2. Run the model forward
3. Produce logits for possible next tokens
4. Compare predictions with target tokens
5. Calculate cross-entropy loss
6. Backpropagate
7. Update trainable parameters

4.3 Cross-entropy intuition

For target token y :

$\text{Loss} = -\log P(y \mid \text{context})$

Correct token probability = 0.90
-> low loss

Correct token probability = 0.10
-> high loss

4.4 Token-by-token

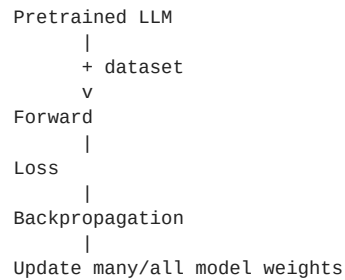
If the target is 'Binary search is efficient', the model is trained to assign high probability to the appropriate next token given the preceding context. This happens across many target positions and many examples.

4.5 Important distinction

SFT is a training approach. LoRA is a parameter-efficient adaptation method. QLoRA combines low-bit base-model storage with LoRA adaptation.

5. Full Fine-Tuning vs PEFT

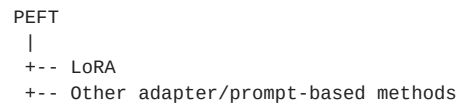
5.1 Full fine-tuning



Full fine-tuning can be very expensive for large models because training requires memory for more than the weights: gradients, optimizer states, activations and temporary tensors.

5.2 Parameter-Efficient Fine-Tuning

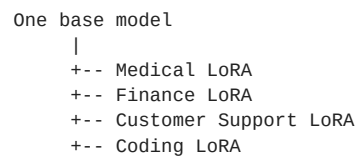
PEFT means Parameter-Efficient Fine-Tuning. The idea is to adapt the model without making every base parameter trainable.



5.3 Why PEFT matters

- Fewer trainable parameters
- Smaller optimizer state for trainable parameters
- Lower training memory pressure
- Often easier to store/share task-specific adapters
- One base model can be reused with multiple adapters

5.4 Adapter reuse



6. LoRA - From Scratch with Mathematics

6.1 The problem

Suppose a transformer layer has a large weight matrix W . Full fine-tuning would learn a full update ΔW . LoRA assumes a useful task-specific update can be represented using a low-rank factorization.

6.2 The equation

Original:

$$y = W x$$

LoRA:

$$y = W x + (\alpha / r) B A x$$

W = frozen pretrained weight

A = trainable low-rank matrix

B = trainable low-rank matrix

r = rank

α = scaling hyperparameter

6.3 Why is it 'low-rank'?

If W is $d_{\text{out}} \times d_{\text{in}}$, a full update has $d_{\text{out}} \times d_{\text{in}}$ parameters. LoRA represents the update as B times A , where the intermediate dimension r is much smaller than d_{in} and d_{out} .

W : $d_{\text{out}} \times d_{\text{in}}$

A : $r \times d_{\text{in}}$

B : $d_{\text{out}} \times r$

BA :

$$(d_{\text{out}} \times r) \times (r \times d_{\text{in}})$$

$$= d_{\text{out}} \times d_{\text{in}}$$

6.4 Numerical example

If W is 4096×4096 , it contains 16,777,216 parameters. With $r=16$, A and B together contain 131,072 parameters. That is a dramatic reduction in trainable parameters for that layer.

	Full update	LoRA, $r=16$
Matrix	4096×4096	$4096 \times 16 + 16 \times 4096$
Parameters	16,777,216	131,072
Trainable idea	Full ΔW	Low-rank ΔW

7. LoRA Hyperparameters and What They Mean

7.1 Rank r

Rank controls the capacity of the low-rank update. Smaller r means fewer trainable parameters. Larger r gives the adapter more capacity but increases cost.

7.2 `lora_alpha`

Alpha is a scaling hyperparameter. In the common LoRA formulation, the update is scaled by α/r .

$$\Delta W = (\alpha/r) B A$$

If $\alpha = 16$ and $r = 16$:
 $\alpha/r = 1$

7.3 `lora_dropout`

Dropout can be applied to the LoRA path as a regularization mechanism. A configuration of 0 means no LoRA dropout is used.

7.4 `bias='none'`

This configuration means biases are not being trained as part of the LoRA setup.

7.5 Why initialize the adapter carefully?

LoRA implementations commonly initialize the adapter path so the initial effective model remains close to the pretrained behavior. The adapter then learns a task-specific update during training.

7.6 What is actually updated?

LoRA training: W stays frozen. A and B receive gradients and are updated. The dataset is only the source of training examples.

7.7 Strong interview answer

Q: Explain LoRA mathematically.

A: For $y=Wx$, LoRA freezes W and learns a low-rank update $\Delta W=(\alpha/r)BA$. Therefore $y=Wx+(\alpha/r)BAx$. Since A and B are much smaller than W , the number of trainable parameters is greatly reduced.

8. Quantization - Why QLoRA Can Fit on Smaller GPUs

8.1 What is quantization?

Quantization represents numerical values using fewer bits. It reduces storage and can reduce memory requirements. The trade-off is quantization error.

8.2 Memory intuition for a 3B model

Representation	Approx bytes/value	3B raw weights (simplified)
FP32	4	12 GB
FP16/BF16	2	6 GB
INT8	1	3 GB
4-bit	0.5	1.5 GB

These are raw-weight estimates, not full training-memory requirements. Real memory is higher because of activations, optimizer states, metadata and runtime overhead.

8.3 Simple quantization math

```
q = round((x - zero_point) / scale)
```

```
x_hat = scale * (q - zero_point)
```

```
x_hat approximates x.
```

8.4 INT8

Signed INT8 has 256 possible integer codes, from -128 to 127. A floating-point range is mapped to this finite set.

8.5 Quantization error

Because a high-precision number is approximated by a limited set of values, x_{hat} is not exactly x . Good quantization tries to keep the resulting model-quality degradation acceptably small.

9. NF4 - NormalFloat 4-bit

9.1 Definition

NF4 stands for NormalFloat 4-bit. It is a 4-bit quantization data type designed to work well with the approximate distribution of pretrained neural-network weights.

9.2 Why 4-bit?

Four bits give $2^4 = 16$ possible codes. Compared with FP16 or FP32, the storage per base weight is much smaller.

```
4 bits
|
+-- 2^4 = 16 codes
|
v
compact base-weight representation
```

9.3 Why 'Normal'Float?

Pretrained neural-network weights are often approximately bell-shaped/normal-like. NF4 uses a normal-distribution-aware codebook rather than simply treating all values as a uniform integer range.

9.4 Classroom explanation

```
Original floating-point weights
|
v
Choose nearby NF4 representable level
|
v
Store a compact 4-bit code
```

Important: A simple classroom implementation that manually defines 16 representative values explains the idea, but it is not the complete production implementation detail of NF4 used by libraries such as bitsandbytes.

9.5 NF4's role

NF4 is about how the frozen base weights are represented. It is not the same thing as LoRA. LoRA is about the trainable adaptation.

10. QLoRA - Put LoRA and Quantization Together

QLoRA = 4-bit quantized frozen base model + trainable LoRA adapters.

10.1 Why QLoRA?

LoRA solves the problem of too many trainable parameters, but the base model can still be large in memory. QLoRA reduces the memory needed for the frozen base by keeping it in a low-bit representation.

```

Pretrained LLM
  |
  v
4-bit quantization / NF4
  |
  v
Frozen base model
  +
LoRA A/B adapters
  |
  v
Training on dataset
  
```

10.2 Mathematical picture

```

W -> W_q

Effective layer:
W_eff ~= W_q + (alpha/r) B A

W_q = frozen quantized base
A,B = trainable adapters
  
```

10.3 What changes and what does not?

Component	Role	QLoRA training
Base LLM weights	General pretrained capability	Frozen
4-bit/NF4 representation	Reduce base memory	Frozen representation
LoRA A/B	Task-specific update	Trainable
Dataset	Training examples	Not trained
Loss	Prediction error signal	Calculated
Optimizer	Applies updates	Updates A/B

10.4 The most important sentence

In QLoRA, the large pretrained base model is kept frozen and quantized; the dataset is used to compute loss, and gradients update only the small LoRA adapter parameters.

11. Double Quantization, Paged Optimizers and Other Memory Tricks

11.1 Double quantization

Quantization itself uses constants/scales/metadata. Double quantization reduces the overhead of those quantization constants by quantizing them as well.

```
Normal:
Weights -> 4-bit values + quantization constants
```

```
Double:
Weights -> 4-bit values
        + quantization constants
        -> quantize those constants too
```

11.2 Paged optimizers

Training may experience GPU memory spikes. Paged optimizers use paging-style memory management so optimizer states can be moved between GPU and CPU memory when necessary.

```
GPU memory
+-----+
| model          |
| activations     |
| optimizer state |
| temporary tensors |
+-----+
      |
      v
CPU memory when needed
```

11.3 Gradient checkpointing

Instead of storing every activation needed for backward propagation, selected activations are stored and some are recomputed later. This saves memory at the cost of additional computation.

```
Normal:
forward -> store many activations -> backward
```

```
Checkpointing:
forward -> store selected checkpoints
backward -> recompute some activations -> continue
```

11.4 Gradient accumulation

If a GPU can only fit a small micro-batch, gradients can be accumulated over several micro-batches before one optimizer update.

```
micro-batch 1 -> accumulate
micro-batch 2 -> accumulate
micro-batch 3 -> accumulate
micro-batch 4 -> optimizer step
```

```
Example:
batch 2 x 4 accumulation steps
~ effective batch 8 on one device
```

12. Optimizer Memory and AdamW

12.1 Why optimizer state matters

Adam/AdamW maintains running statistics in addition to the parameters themselves. During full fine-tuning, these states can be large because many parameters are trainable.

$$\begin{aligned}m_t &= \text{beta1} * m_{(t-1)} + (1 - \text{beta1}) * g_t \\v_t &= \text{beta2} * v_{(t-1)} + (1 - \text{beta2}) * g_t^2\end{aligned}$$

The optimizer uses these statistics to produce parameter updates.

12.2 8-bit optimizer

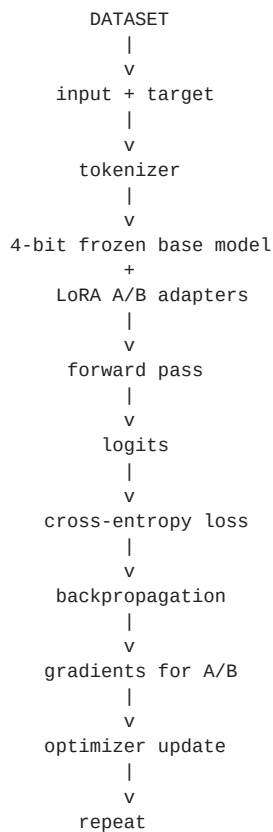
An 8-bit optimizer configuration can reduce the memory used to represent optimizer state. This is separate from model-weight quantization.

Thing	Precision/concept	Purpose
Base model in QLoRA	4-bit, e.g. NF4	Reduce base-weight memory
Optimizer	Can use 8-bit state	Reduce optimizer memory
LoRA parameters	Trainable parameters	Learn task-specific update

12.3 Do not confuse these

4-bit model loading is not the same as an 8-bit optimizer. NF4 is not LoRA. Double quantization is not gradient checkpointing. Each solves a different memory/representation problem.

13. Exactly What Happens During One QLoRA Training Step



13.1 Forward pass

The model processes the tokenized input and produces logits over the vocabulary at the relevant positions.

13.2 Softmax

$$P_i = \exp(z_i) / \sum_j \exp(z_j)$$

z_i = logit for vocabulary token i

13.3 Loss

For target token y :

$$L = -\log P(y \mid \text{context})$$

13.4 Backpropagation in QLoRA

The loss is differentiated with respect to the trainable adapter parameters. The base quantized weights remain frozen.

$$\begin{aligned} \text{grad_A} &= dL/dA \\ \text{grad_B} &= dL/dB \end{aligned}$$

```
A <- optimizer_update(A, grad_A)
B <- optimizer_update(B, grad_B)
```

W_q stays frozen.

14. Llama + Unsloth Style Code: Read It Like a Human

14.1 Loading a pretrained model

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Llama-3.2-3B-Instruct",
    max_seq_length=2048,
    dtype=dtype,
    load_in_4bit=True,
)
```

Meaning: load an already pretrained Llama 3.2 3B Instruct model, prepare for the chosen sequence length, use an appropriate compute datatype, and load the base model in 4-bit form.

14.2 Attaching LoRA

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
)
```

Meaning: add trainable low-rank adapters to selected transformer projections. The base model remains frozen for LoRA training.

14.3 What are q_proj, k_proj, v_proj and o_proj?

They are important linear projections in the self-attention block. Q, K and V are used to construct attention; o_proj maps the attention result back into the model representation.

14.4 What are gate_proj, up_proj and down_proj?

They belong to the feed-forward/MLP portion of the transformer. Adding LoRA there gives the adapter more capacity to modify representations.

14.5 Why use Unsloth?

Unsloth is an optimization toolkit for efficient LLM fine-tuning and inference. In a practical workflow it helps with model loading, PEFT/LoRA setup and memory/computation efficiency.

15. Dataset Formatting, SFTTrainer and Hyperparameters

15.1 Why format the dataset?

Raw data may contain separate columns such as problem, response, solution or context. For causal-LM SFT, these fields are usually converted into a consistent text template so the model sees the intended instruction-response structure.

Example:

You are an assistant.

```
<problem>
{problem}
</problem>
```

```
<response>
{desired_response}
</response>
```

15.2 Tokenization

```
text
|
v
tokenizer
|
+--> input_ids
+--> attention_mask
+--> labels (depending on training setup)
```

15.3 SFTTrainer

```
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    ...
)

trainer.train()
```

The trainer automates the repeated training loop: batching, forward pass, loss, backward pass, optimizer step, logging and checkpointing according to configuration.

15.4 Hyperparameters

Parameter	Meaning
r	LoRA rank; adapter capacity and size
lora_alpha	LoRA scaling
lora_dropout	Regularization on LoRA path
learning_rate	Magnitude of optimizer updates
batch size	Examples in one micro-batch per device
gradient accumulation	Micro-batches before optimizer update
max_seq_length	Maximum sequence length used
max_steps	Maximum training steps when specified
gradient checkpointing	Save activation memory by recomputation
adamw_8bit	Reduce optimizer-state memory

16. Training vs Inference

16.1 Training

```
Input
|
Model
|
Prediction
|
Loss
|
Backpropagation
|
Update trainable parameters
```

16.2 Inference

```
Input
|
Model
|
Generate token
|
Generate next token
|
Final response

No optimizer update.
No normal training backpropagation.
```

16.3 Temperature

A conceptual temperature transformation is $\text{softmax}(z/T)$. Lower temperature makes the distribution sharper and tends to make generation more deterministic. Higher temperature makes it more varied.

16.4 Saving adapters

LoRA allows a small adapter to be stored separately from the base model. This is useful when many task-specific adapters share one base model.

16.5 GGUF/local serving

GGUF is a model file format commonly used by local inference ecosystems such as llama.cpp-based tooling. A workflow can export a compatible representation for local inference.

17. How QLoRA Is Used in a Real System

17.1 Example: customer-support assistant

Business requirement:

"Answer customer questions in our preferred format, tone and workflow."

1. Select a capable pretrained LLM
2. Collect high-quality support examples
3. Clean and validate data
4. Create instruction -> response examples
5. Load base model in 4-bit
6. Add LoRA adapters
7. Train with SFT
8. Evaluate on held-out data
9. Save adapter/model
10. Deploy inference service
11. Monitor quality and safety

17.2 What should the dataset contain?

- Realistic user questions
- Correct and high-quality desired responses
- Consistent format and policy behavior
- Hard cases and edge cases
- Examples that resemble the production input distribution

17.3 Why data quality matters

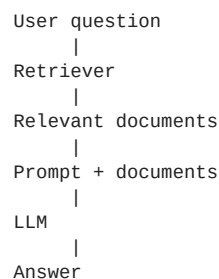
The model learns from the examples. If examples are inconsistent, incorrect or undesirable, fine-tuning can teach those patterns. A good fine-tuning pipeline therefore includes cleaning, validation, deduplication and careful train/validation/test separation.

17.4 Evaluation

- Compare the fine-tuned model against the original base model.
- Use a held-out validation/test set.
- Measure task-specific metrics where possible.
- Test edge cases and formatting.
- Use human evaluation for subjective qualities such as helpfulness/style when appropriate.

18. Fine-Tuning vs RAG

18.1 RAG



RAG retrieves external information at inference time. It is especially useful when information changes frequently or must be grounded in documents.

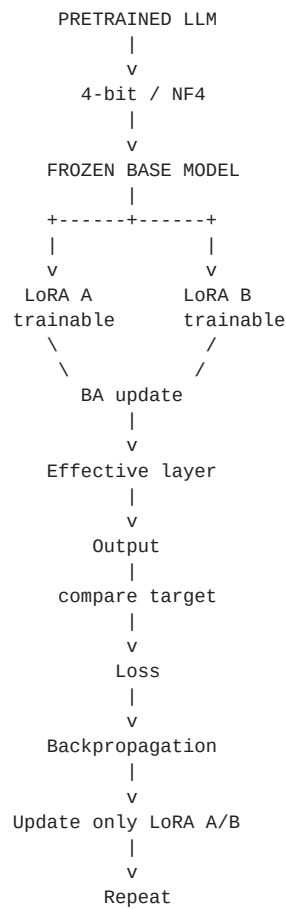
18.2 Fine-tuning

Fine-tuning changes learned parameters or adapter parameters so the model behaves differently after training.

Requirement	Often better starting point
Latest company policy	RAG
Changing product catalog	RAG
Private documents that should be cited	RAG
Consistent output style/format	Fine-tuning can help
Repeated specialized task behavior	Fine-tuning can help
Behavior + current knowledge	Fine-tuning + RAG

Easy rule: Frequently changing knowledge -> think RAG. Behavior/style/task adaptation -> think fine-tuning/LoRA. This is a rule of thumb; real systems often combine both.

19. Complete QLoRA Mental Model



What each part means

Part	Meaning
Pretrained LLM	Already learned general language/model representations.
Dataset	Examples showing the desired task/behavior.
Fine-tuning	Additional training using those examples.
LoRA	Small low-rank trainable update.
Quantization	Represent values using fewer bits.
NF4	4-bit quantization format for neural-network weights.
QLoRA	Quantized frozen base + LoRA adaptation.
Loss	Numerical prediction error.
Gradient	Derivative information for parameter updates.
Optimizer	Uses gradients/state to update trainable parameters.

20. Interview Questions - Strong Answers

Q1. What is fine-tuning?

Answer: Fine-tuning is additional training of a pretrained model on focused data so it adapts to a particular task, domain, output format or behavior.

Q2. If the LLM is already trained, why do we need a dataset?

Answer: The pretrained model has general capabilities, but the new dataset provides examples of the specific behavior we want. Those examples produce loss, and the loss drives parameter updates.

Q3. Are we training the dataset?

Answer: No. The dataset is the training signal. The model parameters are updated. In LoRA/QLoRA, the LoRA adapter parameters are the trainable part.

Q4. What is LoRA?

Answer: LoRA is a parameter-efficient fine-tuning method. It freezes the base weight W and learns a low-rank update $\Delta W = (\alpha/r)BA$.

Q5. What is QLoRA?

Answer: QLoRA combines a low-bit, commonly 4-bit, frozen base model with trainable LoRA adapters. Quantization reduces base-model memory while LoRA reduces the number of trainable parameters.

Q6. What is NF4?

Answer: NF4 is a 4-bit NormalFloat representation designed around the approximate distribution of neural-network weights and commonly used for QLoRA-style loading.

Q7. Why not full fine-tuning?

Answer: Full fine-tuning is expensive because gradients, optimizer states and activations add large memory requirements. LoRA reduces trainable parameters, and QLoRA also reduces base-model weight storage.

Q8. What is LoRA rank?

Answer: Rank r is the intermediate dimension in the low-rank matrices. It controls adapter capacity and trainable parameter count.

Q9. What is lora alpha?

Answer: Alpha is a scaling hyperparameter; in the common formulation the LoRA update is scaled by α/r .

Q10. What does the dataset do in SFT?

Answer: It provides input-target examples. The model predicts target tokens, loss measures the mismatch, and gradients update trainable parameters.

21. Interview Questions - Part 2

Q11. What is the difference between training and inference?

Answer: Training includes forward pass, loss, backward pass and parameter updates. Inference generates predictions without optimizer updates.

Q12. What is gradient checkpointing?

Answer: It saves activation memory by storing selected checkpoints and recomputing some intermediate activations during backward propagation, trading memory for compute.

Q13. What is gradient accumulation?

Answer: It accumulates gradients over several micro-batches before an optimizer step, allowing a larger effective batch size with less memory per micro-batch.

Q14. Why q proj, k proj, v proj and o proj?

Answer: They are important linear projections in the attention mechanism. LoRA can learn task-specific changes in those transformations.

Q15. Why gate proj, up proj and down proj?

Answer: They are projections in the transformer MLP/feed-forward block, giving LoRA additional capacity to adapt representations.

Q16. What is double quantization?

Answer: It reduces memory overhead by quantizing the constants/metadata used by the first quantization step.

Q17. What are paged optimizers?

Answer: They use paging-style memory management for optimizer state and can move state between GPU and CPU memory to handle memory pressure/spikes.

Q18. What is PEFT?

Answer: Parameter-Efficient Fine-Tuning is a family of approaches that adapt a model without updating all base parameters. LoRA is one example.

Q19. Fine-tuning or RAG for changing company policies?

Answer: Usually RAG, because the policy can change and retrieval can use the current source document. Fine-tuning is more appropriate for behavior/style/task adaptation.

Q20. What exactly changes in QLoRA?

Answer: The base quantized model stays frozen. The LoRA adapter parameters A and B are updated using gradients produced from the training loss.

22. Common Mistakes to Avoid

Wrong statement	Correct understanding
We train the dataset.	We train/update model or adapter parameters using the dataset.
QLoRA trains the 4-bit base weights.	The quantized base is frozen; LoRA adapters are trained.
NF4 and LoRA are the same.	NF4 is a quantization format; LoRA is an adaptation method.
8-bit Adam means the model is 8-bit.	Optimizer precision and model-weight precision are separate.
Fine-tuning is best for current facts.	For changing facts, RAG is often better.
4-bit means every computation is 4-bit.	Storage precision and compute precision are distinct.
More LoRA rank is always better.	Higher rank increases capacity and cost; evaluate it.
Training loss proves quality.	Use held-out and task-specific evaluation.
Dataset directly changes weights.	Dataset -> prediction -> loss -> gradients -> update.

Most important correction: Do not say 'the dataset teaches the model directly.' Say: 'The dataset provides examples; the loss and gradients derived from those examples cause trainable parameters to update.'

23. Teaching Script - How to Explain This to Someone

If you need to teach fine-tuning and QLoRA, do it in this order. Start conceptually, then add mathematics.

Step 1 - Start with the pretrained LLM

Say: 'We already have a trained LLM. It has general language capability stored in billions of learned parameters.'

Step 2 - Introduce the new dataset

Say: 'Now our application needs a particular behavior. We collect examples of inputs and the answers we want. This dataset is like a teacher or practice book.'

Step 3 - Explain prediction and loss

Say: 'The model tries to predict the target tokens. We compare its probability distribution with the target and calculate loss.'

Step 4 - Explain learning

Say: 'Backpropagation calculates gradients. The optimizer uses those gradients to change trainable parameters.'

Step 5 - Explain full fine-tuning

Say: 'If we make the whole LLM trainable, memory and compute can become very expensive.'

Step 6 - Introduce LoRA

Say: 'Instead of changing the entire weight matrix, LoRA freezes the base and learns a small low-rank update A and B.'

Step 7 - Introduce quantization

Say: 'The frozen base model can still consume a lot of memory, so we store its weights using fewer bits.'

Step 8 - Introduce QLoRA

Say: 'QLoRA combines the two ideas: a quantized frozen base and trainable LoRA adapters.'

Step 9 - Explain NF4 last

Say: 'NF4 is one of the 4-bit representations used to store neural-network weights efficiently.'

Step 10 - End with one diagram

```
Dataset
-> tokenize
-> 4-bit frozen base + LoRA
-> prediction
-> loss
-> backprop
-> update LoRA
-> repeat
-> evaluate
-> deploy
```

24. Real Project Checklist

Before training

- Define the exact task/behavior you want to improve.
- Decide whether the problem needs RAG, fine-tuning or both.
- Choose a base model with suitable general capability.
- Collect high-quality input-output examples.
- Clean duplicates, contradictions and bad samples.
- Create train/validation/test splits.
- Define evaluation metrics before training.

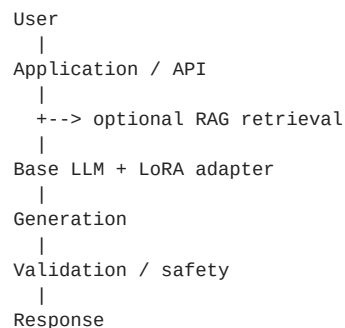
During training

- Load the base model with an appropriate quantization/precision strategy.
- Attach LoRA to selected target modules.
- Choose rank, alpha, dropout and learning rate.
- Use gradient checkpointing/accumulation if memory is limited.
- Monitor training and validation behavior.
- Watch for overfitting and degraded general behavior.

After training

- Compare against the original base model.
- Test real user-like prompts and edge cases.
- Check formatting, safety and reliability.
- Save the adapter/model artifacts.
- Choose a serving format/runtime.
- Monitor production quality and collect feedback.

Production architecture



25. One-Page Interview Cheat Sheet

Term	One-line meaning
LLM	Large neural language model with learned parameters.
Parameter/weight	Numerical value learned during training.
Pretraining	Initial large-scale training for broad capability.
Fine-tuning	Further training of a pretrained model on focused data.
SFT	Supervised fine-tuning using desired outputs as targets.
PEFT	Parameter-efficient fine-tuning family.
LoRA	Low-rank trainable update with frozen base weights.
Quantization	Represent numerical values with fewer bits.
NF4	4-bit NormalFloat representation for neural-network weights.
QLoRA	Quantized frozen base + LoRA adapters.
Double quantization	Quantize quantization constants to reduce overhead.
Paged optimizer	Paging-style optimizer-state memory management.
Gradient checkpointing	Save activation memory by recomputation.
Gradient accumulation	Accumulate micro-batch gradients before update.
Loss	Measures prediction error.
Gradient	Derivative information used for updates.
AdamW	Common optimizer with Adam-style moments and decoupled weight decay.
RAG	Retrieve external/current information at inference time.

Core equations

Pretraining/full update:
 $W_{\text{new}} = W_{\text{old}} - \eta * \text{grad}_W(L)$

LoRA:
 $\Delta W = (\alpha/r) B A$

Effective layer:
 $W_{\text{eff}} = W + (\alpha/r) B A$

QLoRA intuition:
 $W \rightarrow W_q$ (quantized, frozen)
 $W_{\text{eff}} \approx W_q + (\alpha/r) B A$

Attention:
 $\text{softmax}(QK^T / \sqrt{d_k}) V$

Cross-entropy:
 $L = -\log P(y | \text{context})$

Final memory line: Pretrained LLM = general capability. Dataset = examples. Fine-tuning = additional learning. LoRA = small trainable update. QLoRA = LoRA + quantized frozen base.

26. The 60-Second Interview Answer

Question: Explain how you fine-tuned an LLM using QLoRA.

Answer: I started with a pretrained instruction-tuned LLM rather than training from scratch. I prepared a task-specific supervised dataset containing input-output examples. The examples were formatted and tokenized, and during SFT the model predicted target tokens and the loss measured the difference from the desired output. To make training memory-efficient, I loaded the base model in 4-bit form and attached LoRA adapters to selected transformer projection layers. The base model remained frozen, while only the LoRA adapter parameters were optimized through backpropagation. Memory-saving techniques such as gradient checkpointing, gradient accumulation and an efficient optimizer configuration can further reduce memory pressure. After training, I evaluated the adapted model against the base model and prepared the adapter/model for inference or deployment.

If asked: 'What exactly changed?'

Answer: 'The base LLM weights did not change in QLoRA. The trainable LoRA A/B parameters changed. Those learned parameters alter the effective transformations of selected layers and therefore adapt the model's behavior.'

If asked: 'What did the dataset do?'

Answer: 'The dataset did not get trained. It provided supervised examples. Those examples produced target-token loss, and the resulting gradients were used to update the trainable LoRA parameters.'

If asked: 'Why RAG instead?'

Answer: 'For frequently changing external knowledge, I would generally use RAG because it retrieves current source material at inference time. For behavior, style and repeated task adaptation, fine-tuning/LoRA can be useful. A production system can combine both.'

You understand the topic if you can explain this without notes:

Pretrained LLM -> task-specific dataset -> tokenization -> prediction -> loss -> gradients -> update trainable parameters. With LoRA, only A/B adapters are trained. With QLoRA, the large base model is additionally kept in a 4-bit quantized form.

Final revision strategy

First memorize the data-flow, then the LoRA equation, then the QLoRA definition, then the memory-saving techniques, and finally the interview answers. Do not memorize isolated buzzwords without knowing what problem each one solves.